PROGRAMMABLE MONEY · AGENTIC ECONOMY · ARC

Bondwire

Trust and settlement primitives for autonomous agents, settled in USDC on Arc.

LIVE ON ARC TESTNET | CHAIN 5042002 | OWNERLESS CONTRACTS | ALL THREE CONTRACTS EXACT MATCH VERIFIED

github.com/Mnorbert87/bondwire

mnorbert87.github.io/bondwire

THE PROBLEM

Agents can already pay. They still cannot be trusted.

Every agentic payment demo quietly assumes two things that do not exist on chain yet.

MISSING ASSUMPTION 1

You can trust the agent before it touches your money

There is no cost to being a new agent. Reputation without stake is a signup form, and a burned identity is replaced for free.

MISSING ASSUMPTION 2

You can pay it while it works, not after

Invoicing assumes a relationship and a clock both parties accept. Autonomous counterparties have neither.

WHERE THE FIELD STOPS

x402 and most of the category solve one act: pay per call. The open question is the one after the payment leaves. **Who pays when the agent defaults, and who pays when the verifier lies?** That is the layer Bondwire ships.

THE PRODUCT

Three ownerless primitives that compose

No owner, no admin role, no proxy, no pause. The only roles are the ones in the protocol itself, scoped per obligation.

TRUST LAYER

AgentBond

An agent deposits USDC as skin in the game. Any enforcer it approves can lock a slice behind an obligation and slash it on default. Free bond is a money backed credit score.

0x4383…702c

SETTLEMENT LAYER

StreamPay

Continuous USDC settlement. Funds accrue per second, the recipient withdraws any time, either side cancels with a fair split. Pay per second of work, not per invoice.

0x6C2A…B262

MECHANISM

CommitStakeV2

Pay only on verified PASS. The verifier posts its verdict with its own bond slice locked behind it. A challenge window and an optional arbiter give recourse. Lying costs real money.

0xf3457ABf…af1474

WHY COMPOSABLE MATTERS

The two primitives stand alone. Together they form a complete trust and pay rail: a stream can fund a bond, a fulfilled commitment can release one, and a false verdict burns a bonded slice. That composition is only free and permissionless on a shared ledger.

PROOF

Every claim here is a transaction you can open

Not a Live badge over a mock. The anti collusion mechanism has fired on chain, twice.

WHAT	AMOUNT	ON CHAIN
Overturn burn, the surplus left after the harmed party is paid	1.45 USDC	0xf46f062d…3e9d1d · Transfer to 0x…dEaD
Liveness burn, the whole slice when a dispute deadlocks	1.50 USDC	0xae25f951…b5a364 · Transfer to 0x…dEaD
Cross chain capital onboarding, Base Sepolia to Arc via CCTP V2	1 USDC	3 transactions, all status 1, bond 3 to 4
Source on arcscan, byte for byte with this repo	exact match, all three	fully verified, not matched from a bytecode database

THE TESTS BITE, AND WE MEASURED IT RATHER THAN COUNTING THEM

A green number proves nothing on its own, so both fixes on the open branch were mutation checked: **removing the three new ceilings fails four tests, and reverting the helper to ignore the arbiter fee fails the regression test and the new fuzz property, which finds its counterexample even at a single run.** That is what says the suite would catch the bug coming back.

HOW IT WORKS

One agent lifecycle, settled end to end in USDC

01

Bond up

The agent deposits USDC. Free bond becomes a public, slashable credit score.

`AgentBond`

02

Get hired

A counterparty reads the bond and prices the risk before any work starts.

`off chain`

03

Lock collateral

An agent authorized enforcer locks a slice behind the job as an obligation.

`AgentBond`

04

Get paid per second

A stream runs while the work runs. The recipient withdraws any time.

`StreamPay`

05

Verified PASS

A bonded verifier posts the verdict with its own slice at risk. Challenge window, then arbiter.

`CommitStakeV2`

06

Release or slash

Fulfilled releases the bond. Default routes it to the creditor. A lie burns the verifier's slice.

`composed`

Distinct wallets, not self dealing

Agent, verifier and arbiter are separate addresses on chain, and the bond is never resolved by the party it belongs to. On the live AgentBond, obligation #1 was released by its enforcer and #2 was slashed, sending 1.5 USDC to the creditor. Here the enforcer and the creditor are the same address because the enforcer is CommitStakeV2 itself, which is what routes a default to the harmed side. The agent cannot release or slash its own bond.

Nine terminal outcomes, all invariant tested

Clean pass, clean fail, two upheld challenges, two overturns, two arbiter silences, and the liveness slash. Count them in the `Outcome` enum: it has ten members and the first one is `None`, which is not terminal. The solvency invariant holds across all nine: the contract can never owe more than it holds.

ALREADY WIRED IN

It plugs into the stack agents actually use

CIRCLE CCTP V2 · BRIDGE KIT

Capital does not have to start on Arc

1 USDC on Base Sepolia, bridged with Bridge Kit and deposited straight into AgentBond. Arc is already in the chain list, CCTP domain 26, so the cross chain leg is one kit.bridge() call. Three transactions, all status 1, runnable with npm run onboard.

ERC-8004

Real on chain identity, plus the money layer it lacks

Both demo agents hold identity NFTs in Arc's IdentityRegistry, with real third party feedback through ReputationRegistry and a separate execution wallet bound over EIP-712. ERC-8004 says who an agent is. It does not say what happens to the money when the agent lies. That is the gap we fill.

X402

Pay per call, settled per second underneath

A runnable demo gates an HTTP API behind a live StreamPay stream. The agent pays over HTTP 402 and the settlement is continuous rather than per invoice.

SDK · MCP SERVER

An agent can bond from inside its own tool loop

Single file ethers v6 wrapper with human USDC amounts, and an MCP server so any AI agent can check a passport, post a bond and open a bonded escrow, with quote before execute on every value moving call.

CIRCLE WALLETS

The agent signs without ever holding a key

A second binding runs the same lifecycle through a Circle developer controlled wallet: approve, deposit, open a stream, withdraw. Each step goes out as createContractExecutionTransaction and is polled to a terminal state. No private key sits on disk. The approve leg is on chain as 0x7ec0f1...0a28.

WHAT IS SHIPPED

Not a prototype. A deployed, tested, documented stack.

4
contracts live on Arc testnet, ownerless, source verified

226
Solidity tests: unit, adversarial, cold audit, fuzz

5
Halmos symbolic proofs over the money paths

150k
calls per invariant campaign (10k runs × depth 15), zero solvency violations

9
static dApp pages, no backend, live RPC reads

3
runnable demos: commerce, x402, CCTP

84.5%
measured mutation score, survivors classified one by one

0
admin keys, proxies or pause switches

AND THE DOCUMENTATION A REVIEWER ACTUALLY NEEDS

THREAT_MODEL, SECURITY_AUDIT, VERIFIER_ECONOMICS, TEST_AUDIT, HALMOS_VERIFICATION, MUTATION_TESTING and a two minute JUDGES walkthrough. CI is green on every push and runs the same command a judge would.

HONESTY AS A FEATURE

What is still wrong with it, in our own words

Three findings were fixed on public branches and, on 2026-07-31, **deployed**. Holding them back would have preserved the old deployment's exact match verification, so we redeployed and re verified instead: all three contracts are exact match again, at new addresses. We measured that rather than assumed it. Two are **still open**, and they belong on this slide: one called what is still wrong should contain something that still is.

FINDING	STATUS	BRANCH
Sockpuppet arbiter can reclaim the verifier's slice	Fixed and tested, opt in griefing guard	`fix/arbiter-griefing-optin`
AgentBond allowance race, revoke is not final	Fixed and tested, allowance epochs	`fix/agentbond-allowance-epoch`
Staker chosen time parameters were unbounded	Fixed and tested, three ceilings, 125 tests	`fix/commitstake-param-bounds`
USDC blacklist can strand a terminal transition	**Open, by choice.** Payouts are pushes and USDC can blacklist. Medium: counterparties are pinned at create. The fix touches every payout path, not a rushed change	`THREAT_MODEL §9`
Aggregate leverage is unbounded, only per commitment leverage is capped	**Open.** N commitments stack, so a third of a bond in capital can pin all of it. The revoke exit is real, which is what fix 2 bought	`AUDIT_SUMMARY`

Self correction, in public

Section 8 of the security document claimed the sockpuppet arbiter did not profit. It was wrong. We corrected it in the repo and on the page people actually read, rather than quietly deleting the claim.

THE LIMIT OF OUR OWN EVIDENCE

The invariant campaign bounds deadlines to thirty days, so a green campaign was never evidence against the unbounded lock finding. We wrote that into the public threat model as a limit of the thing we are otherwise proud of.

USDC as gas is what makes this economically possible

The chain is part of the design

- **One asset end to end.** Gas, bond, stream and slash are all USDC. An agent holds no volatile second token just to be able to act.
- **Micro settlement stays viable.** Per second streams and dust sized slices only make sense where fees are stable and denominated in the same unit as the value.
- **Neutral settlement between distrusting parties.** A shared database needs a trusted operator. These agents do not trust each other, so the ledger, not a party, has to be the trusted one.

Roadmap

- **Uncollateralized agent credit.** The headline use case: early agents borrow the bond against future cash flow. These primitives are the enforcement substrate that makes that recoverable.
- **Verifier set, not a verifier.** Stake backed set, dispute window, slashing for provably wrong attestations. The role is already an isolated address, so it is an interface change, not a redesign.
- **Enforcer accountability.** The same primitive turned on itself: the enforcer posts a bond and is slashable for a wrongful slash.
- **Mainnet.** The three fixes already shipped and were re verified on testnet on 2026-07-31. Carrying them to a mainnet deploy is a separate decision, not a pending code change.

CHECK IT YOURSELF IN TWO MINUTES

Open a transaction. Run the suite. Nothing here needs your trust.

REPOSITORY
github.com/Mnorbert87/bondwire

LIVE DAPPS, NO BACKEND
mnorbert87.github.io/bondwire

BONDED VERIFIER FLOW, REAL TRANSACTIONS
mnorbert87.github.io/bondwire/bonded-verifier

TWO MINUTE JUDGE WALKTHROUGH
JUDGES.md

```
git clone https://github.com/Mnorbert87/bondwire
cd bondwire/contracts/commit-stake-v2 && forge test -vv
```